The case on line 2 is necessary to determine the phone number of the contact based on whether an SMS message is incoming or outgoing. This is purely an application-level concern that will make sense when we write the SmsMessageStore.

The Contact.changeset/1 function is used on line 8 to create a changeset for our contact record. Then, that changeset is inserted into the database on line 10.

The function Repo.insert/2 is called with options that tell Ecto what to do if a duplicate contact occurs. This happens when a contact with the phone number already exists and the unique constraint is violated.

The insert pattern in this example is called an upsert (update or insert) and is natively supported by PostgreSQL and other databases. It's a bit complex with many different options and strategies for dealing with the uniqueness conflict. Ecto has great documentation[17] that goes through many different scenarios and outcomes.

Let's write the Query.SmsMessageStore next.

Query and Persist SMS Messages

We'll do more advanced things in the SmsMessageStore. Our store needs to insert new SMS messages, update the status of existing messages, return messages for a given contact, and load a list of messages for the sidebar view. We'll start with the create function because it's similar to what we just used in the ContactStore.

Create lib/phone_app/conversations/query/sms_message_store.ex and add the following code:

phone_app/lib/phone_app/conversations/query/sms_message_store.ex

```
defmodule PhoneApp.Conversations.Query.SmsMessageStore do
  import Ecto.Query

  alias PhoneApp.Repo
  alias PhoneApp.Conversations.Schema.SmsMessage
  alias PhoneApp.Conversations.Query.ContactStore
end
```

These aliases remove the need for us to type long module paths through our module. The line import Ecto.Query is new. We'll come back to this when we write a query. For now, add the create function:

17. https://hexdocs.pm/ecto/Ecto.Repo.html#c:insert/2-upserts